

On the Imperceptibility of Graph Backdoor Trojans in Spectral Domain

TODO: Author Names

Abstract—Backdoor attacks on graph classification implant a hidden behavior through poisoned training graphs: the trained model classifies clean graphs normally but assigns graphs that carry an attacker’s trigger to a target class. To avoid detection, attacks keep triggers small, so that poisoned graphs resemble clean ones. This paper asks whether structural triggers also remain imperceptible in the spectral domain, that is, whether the normalized-Laplacian spectrum of an individual triggered graph can be distinguished from the spectra of clean graphs. Each graph is summarized by a fixed-length descriptor of its spectrum, which makes graphs of different sizes comparable. Two detectors are built on this descriptor: a clean-reference statistical test, which asks whether a graph deviates from the natural variation of clean spectra, and a multilayer perceptron (MLP) trained on labelled clean and triggered spectra. On four benchmarks with three structural attacks against a graph convolutional network, the statistical test flags most triggered graphs only on MUTAG and for one attack on AIDS, whereas the MLP reaches ROC-AUC values of 0.84–1.00. Rejecting the graphs flagged by the MLP removes nearly all success of a learned trigger on the three molecular benchmarks while rejecting fewer than 10% of clean graphs; on PROTEINS, substantial attack success remains. Because the MLP is trained on examples of the attack it detects, these results measure how much the spectrum reveals about a known attack rather than the performance of a defense against unseen attacks.

Index Terms—Backdoor attacks, graph neural networks, graph signal processing, normalized Laplacian, spectral detection.

I. INTRODUCTION

GRAPH neural networks [1], [2] are widely used to classify graph-structured data, but they can be compromised by backdoor attacks introduced through poisoned training data [3], [4]. In graph classification, an attacker embeds a trigger in selected training graphs and relabels them with an attacker-chosen target class. The trained model then behaves normally on clean graphs but assigns triggered graphs to the target class. Consequently, high accuracy on clean test data cannot establish that a model is free of a backdoor.

To avoid detection, attacks keep triggers imperceptible: they limit how much a trigger may change a graph, so that poisoned graphs resemble clean ones, and some attacks, such as DPSBA [5], explicitly constrain the structural and semantic deviation of poisoned graphs. The attacks evaluated in this paper, for example, confine the trigger to four nodes of the graph. A structural trigger nevertheless rewires edges, and the eigenvalues of the normalized graph Laplacian summarize how a graph is connected [6]. A rewiring that is small in the graph may therefore still leave a trace in its spectrum. This paper asks whether structural triggers remain imperceptible in the

spectral domain: can the spectrum of an individual triggered graph be distinguished from the spectra of clean graphs?

We answer this question in three steps, each building on the previous one. First, a statistical test compares the spectrum of each graph with the variation among clean graphs and asks whether the changes caused by a trigger exceed this natural variation. Second, a multilayer perceptron (MLP) trained on labelled clean and triggered spectra asks whether the spectrum as a whole separates individual triggered graphs from clean ones, even where no single spectral feature does. Third, graphs flagged by either detector are rejected before they reach the trained target model, and we measure how much attack success remains. Throughout, both detectors are compared at approximately the same rate of rejected clean graphs.

The contributions are: (i) a clean-reference test of whether the changes in the normalized-Laplacian spectrum caused by structural backdoor triggers exceed the natural spectral variation among clean graphs; (ii) a spectrum-only MLP detector for individual graphs that requires no access to the target model; and (iii) an evaluation of how rejecting detected graphs affects attack success. On four benchmarks, the MLP separates triggered from clean graphs far better than the statistical test, and rejecting the graphs it flags nearly eliminates the learned trigger on three of them; on the fourth, a substantial part of the attack success remains.

Section II reviews related attacks and defenses. Section III introduces the notation, the backdoor threat model, the structural triggers, and the spectral representation of a graph. Section IV presents the two detectors, and Section V reports the experiments.

II. RELATED WORK

Structural graph backdoors differ in how their triggers are constructed. The subgraph-based backdoor attack (SBA) inserts a reusable random-graph template [3], and Motif-Backdoor inserts a motif at host nodes chosen with the help of a model [7]. In both, the trigger pattern is fixed, although its placement may depend on the host graph. The graph Trojan attack (GTA) instead learns a generator that produces a graph-specific trigger [4], and TRAP perturbs edges guided by gradients [8].

Backdoor attacks also differ in which parts of the graph they perturb. DPSBA constrains the structural and semantic deviation of poisoned graphs through distribution-aware clean-label poisoning [5]. Other attacks leave the structure untouched and perturb only node features: SPEAR preserves the topology [9], and spectral-domain attacks embed the trigger in the spectral coefficients of node-feature signals [10]. Since such feature-only attacks leave the adjacency matrix unchanged, they also

leave the normalized-Laplacian spectrum unchanged and are outside the scope of this paper.

Existing graph backdoor defenses differ mainly in what they need access to. GSP purifies training graphs by spectral low-pass filtering [11], and RIGBD [12] and DShield [13] inspect the possibly poisoned training data and intervene during training. DMGNN prunes triggers using counterfactual explanations of the model [14], and GraphProt avoids access to model internals but queries the classifier repeatedly on sampled subgraphs [15]. A2GBD combines active node selection with reinforcement learning to avoid assumptions about the attack [16]. SSP also relies on an eigenanalysis, but of the feature covariance and combined with neighborhood feature deviations [17], not of the graph Laplacian. The detector studied here needs less: it reads only the normalized-Laplacian spectrum of an incoming graph. Once trained, it requires neither the training set of the target model nor its parameters, gradients, or outputs, and it neither modifies the input graph nor retrains the model; it acts as a filter in front of the model. Its training, however, requires triggered examples of the attack, a requirement discussed in Section IV-B.

III. PRELIMINARIES AND PROBLEM FORMULATION

A. Graph Classification

A graph $G = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ has $n = |\mathcal{V}|$ nodes, an edge set $\mathcal{E}$, an adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, a diagonal degree matrix $\mathbf{D}$ whose entry d_v is the number of neighbors of node v , and a feature matrix $\mathbf{X}$ whose row $\mathbf{x}_v$ is the feature vector of node v . Let $\mathcal{G}$ be the set of such graphs and $\mathcal{C}$ a set of C class labels. A graph classifier $f_\theta: \mathcal{G} \rightarrow \Delta^{C-1}$ with parameters θ maps a graph to a vector of class probabilities, where Δ^{C-1} is the set of probability vectors of length C , and predicts the label $h_\theta(G) = \arg \max_{c \in \mathcal{C}} f_{\theta,c}(G)$.

B. Holm Procedure

The statistical detector of Section IV-A tests each graph at many coordinates of its spectral descriptor at once. Even for a clean graph, some of these tests would reject by chance, so the tests of one graph are corrected for multiplicity with Holm's step-down procedure [18]. For a prescribed significance level α and m p -values sorted in increasing order, the procedure compares the smallest with α/m , the second smallest with $\alpha/(m-1)$, and so on, and rejects all hypotheses before the first comparison that fails. For valid p -values, the probability of rejecting at least one true hypothesis is then at most α , without requiring the tests to be independent. In this paper, only whether a graph has at least one rejection matters. This is decided by the first comparison alone: some hypothesis is rejected exactly when the smallest p -value is at most α/m , and the smallest Holm-adjusted p -value is m times the smallest p -value, capped at 1.

C. Backdoor Trojan

1) *Basics of a Backdoor Trojan*: A backdoor Trojan (backdoor, for short) is a hidden behavior implanted in a model

through poisoned training data and activated by a trigger, a pattern that the attacker inserts into an input graph. The attacker describes the insertion by a trigger policy $T: \mathcal{G} \times \mathcal{C} \rightarrow \mathcal{G}$, which maps a graph G and a target label $y_t \in \mathcal{C}$ to the triggered graph $\tilde{G} = T(G, y_t)$.

Definition 1 (Backdoor attack): Let $\mathcal{P}$ be a distribution over $\mathcal{G} \times \mathcal{C}$, let f_θ be a classifier trained on clean samples from $\mathcal{P}$, and let $\delta_a, \delta_c \in [0, 1)$ be tolerances. A backdoor attack with target label y_t and trigger policy T produces a poisoned classifier f_{θ^*} that satisfies

$$\begin{aligned} \Pr_{(G, y) \sim \mathcal{P}} [h_{\theta^*}(T(G, y_t)) = y_t \mid y \neq y_t] &\geq 1 - \delta_a, \\ \Pr_{(G, y) \sim \mathcal{P}} [h_{\theta^*}(G) = h_\theta(G)] &\geq 1 - \delta_c. \end{aligned}$$

The first condition is the backdoor behavior on triggered inputs, and the second is the preservation of clean behavior. The empirical counterparts of the two conditions, the attack success rate and the clean accuracy measured on test graphs, are defined in Section V-A.

2) *Backdoor Insertion Framework*: A backdoor attack proceeds in three stages [3], [4]: the attacker designs the trigger policy, poisons part of the training data with it, and releases the data to the owner of the target model, whose training implants the backdoor. Let $\mathcal{D} = \{(G_i, y_i)\}_{i=1}^N$ be the clean training set.

a) *Trigger-Insertion Policy Design*: Imperceptibility enters at this stage. With a surrogate classifier $f_{\hat{\theta}}$, the attacker seeks a policy that triggers the target label while changing each graph only slightly:

$$\begin{aligned} \max_T \quad & \frac{1}{N} \sum_{i=1}^N \mathbf{1}[h_{\hat{\theta}}(T(G_i, y_t)) = y_t] \\ \text{s.t.} \quad & d(T(G_i, y_t), G_i) \leq \epsilon, \quad i = 1, \dots, N, \end{aligned} \quad (1)$$

where d measures how much a trigger changes a graph and ϵ bounds this change. When T is a neural network T_ϕ with weights ϕ , problem (1) can be relaxed by replacing the constraint with a penalty d weighted by $\gamma > 0$ and the indicator with the cross-entropy loss, so that ϕ can be trained by gradient descent. The trained generator then produces the trigger for a new graph in a single forward pass. In either form, d is measured on the graph itself. The question of this paper is whether a trigger that changes the graph only slightly also leaves its spectrum nearly unchanged.

b) *Data Poisoning*: The attacker applies the policy to a subset $\mathcal{D}_p \subset \mathcal{D}$ containing a fraction ρ of the training graphs, relabels these graphs with y_t , and releases the poisoned training set

$$\tilde{\mathcal{D}} = (\mathcal{D} \setminus \mathcal{D}_p) \cup \{(T(G, y_t), y_t) : (G, y) \in \mathcal{D}_p\}. \quad (2)$$

The attacker aims to keep the clean accuracy of the resulting model while increasing its attack success; this trade-off is evaluated empirically.

c) *Model Poisoning*: The owner of the target model trains it on $\tilde{\mathcal{D}}$ as released, with its usual training loss and without any control by the attacker, and obtains the poisoned parameters θ^* .

D. Trigger Insertion in Graphs

The three attacks evaluated here modify the structure of a graph inside a small region $\mathcal{V}_g \subseteq \mathcal{V}$ of $k = |\mathcal{V}_g|$ nodes, the trigger size. The trigger removes a set of edges $\mathcal{E}_{\text{cut}} \subseteq \mathcal{E}$ and inserts a set $\mathcal{E}_g$, so that the triggered graph G has the edge set

$$\tilde{\mathcal{E}} = (\mathcal{E} \setminus \mathcal{E}_{\text{cut}}) \cup \mathcal{E}_g, \quad (3)$$

the same node set $\mathcal{V}$, and possibly modified features on $\mathcal{V}_g$. Attacks on node classification such as UGBA [19] and DPGBA [20] instead inject new trigger nodes; the attacks evaluated here all select existing nodes and therefore preserve n . They differ in how they choose $\mathcal{E}_{\text{cut}}$ and $\mathcal{E}_g$.

1) *Static Triggers*: SBA and Motif-Backdoor insert a fixed pattern $\Gamma = (\mathcal{V}_\Gamma, \mathcal{E}_\Gamma)$: an Erdős-Rényi random graph for SBA [3], and for Motif-Backdoor the motif M44, a triangle with a pendant node attached [7]. A placement map $\iota: \mathcal{V}_\Gamma \rightarrow \mathcal{V}$ chooses the host nodes $\mathcal{V}_g = \iota(\mathcal{V}_\Gamma)$, uniformly at random for SBA and by a model-guided ranking for Motif-Backdoor (Appendix A). The edges among the host nodes are replaced by the pattern, that is, $\mathcal{E}_{\text{cut}} = \mathcal{E}[\mathcal{V}_g]$, the edges with both endpoints in $\mathcal{V}_g$, and $\mathcal{E}_g = \iota(\mathcal{E}_\Gamma)$. Edges between the host nodes and the rest of the graph, as well as all features, are unchanged. The pattern is fixed, but its effect on the spectrum can still vary with the host graph and the placement.

2) *Dynamic Triggers*: GTA [4] generates a trigger that depends on the host graph. For a selected region $\mathcal{V}_g$, a topology generator proposes edges between nodes of $\mathcal{V}_g$, which are added to the graph, and a feature generator perturbs the features of these nodes:

$$\tilde{\mathbf{A}} = \max(\mathbf{A}, \mathbf{A}_\phi(G)), \quad \tilde{\mathbf{X}} = \mathbf{X} + \mathbf{X}_\phi(G). \quad (4)$$

Here $\mathbf{A}_\phi(G)$ is a binary adjacency matrix with nonzero entries only between nodes of $\mathcal{V}_g$, the maximum is taken entry by entry, and $\mathbf{X}_\phi(G)$ is nonzero only in the rows of $\mathcal{V}_g$. GTA therefore adds edges ($\mathcal{E}_{\text{cut}} = \emptyset$) and changes features; the choice of $\mathcal{V}_g$ itself is not learned.

All three attacks thus change the edges of the graph. The spectral representation introduced next depends only on the edges: it reflects $\mathcal{E}_{\text{cut}}$ and $\mathcal{E}_g$ but not the feature perturbation of GTA.

E. Graphs in Spectral Domain

The normalized Laplacian of G is

$$\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}, \quad (5)$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. It is symmetric, so it has n real eigenvalues $0 \leq \lambda_1 \leq \dots \leq \lambda_n \leq 2$ with orthonormal eigenvectors [6]. We call the vector $\boldsymbol{\lambda}(G) = [\lambda_1, \dots, \lambda_n]^\top$ the spectrum of G .

An eigenvector $\mathbf{u}$ of $\mathbf{L}$ assigns a value u_v to every node v . For a graph without isolated nodes and an eigenvector of unit length, the corresponding eigenvalue is

$$\lambda = \mathbf{u}^\top \mathbf{L} \mathbf{u} = \sum_{\{v,w\} \in \mathcal{E}} \left(\frac{u_v}{\sqrt{d_v}} - \frac{u_w}{\sqrt{d_w}} \right)^2. \quad (6)$$

An eigenvalue is therefore small when the values of its eigenvector, divided by the square root of the node degrees,

are similar at the two ends of every edge, and large when they differ. In the language of graph signal processing [21], the eigenvectors are the oscillation modes of the graph and the eigenvalues their frequencies.

The small eigenvalues, or low frequencies, describe how the graph is connected. An eigenvalue 0 arises from an eigenvector that is constant (up to the degree scaling) on a connected component, so the number of zero eigenvalues equals the number of connected components, isolated nodes aside. For a connected graph, a small λ_2 indicates a bottleneck: the graph can be split into two sizeable parts by removing comparatively few edges [6]. The large eigenvalues, or high frequencies, belong to eigenvectors whose values alternate between neighboring nodes. The largest possible value, 2, occurs exactly when a connected component is bipartite, that is, when its nodes can be divided into two groups such that every edge joins the two groups [6]. The spectrum does not determine a graph uniquely, but it summarizes its connectivity: how many separate pieces it has, whether it has a bottleneck, and whether it is bipartite. A trigger that rewires the edges among a few nodes may change this connectivity locally, and with it some eigenvalues.

The spectrum has one eigenvalue per node, so graphs of different size have spectra of different length that cannot be compared entry by entry. We therefore place the eigenvalue λ_i at its relative position $r_i = (i - 1)/(n - 1)$ in the sorted spectrum, so that the first and last eigenvalues are at 0 and 1 for every $n \geq 2$, and connect the points (r_i, λ_i) by straight lines. The resulting nondecreasing curve $\Lambda_G: [0, 1] \rightarrow [0, 2]$ keeps the shape of the spectrum but not its length. Reading it at K equally spaced positions gives the spectral descriptor

$$\mathbf{s}(G) = [\Lambda_G(1/K), \Lambda_G(2/K), \dots, \Lambda_G(1)]^\top \in [0, 2]^K, \quad (7)$$

with coordinates $s_j(G) = \Lambda_G(j/K)$. Position 0 is omitted because $\Lambda_G(0) = \lambda_1 = 0$ for every graph with at least one edge. Since the eigenvalues do not depend on the order in which the nodes are numbered, neither does $\mathbf{s}(G)$, so graphs can be compared without matching their nodes; and since every graph is mapped to the same K positions, graphs of different size can be compared coordinate by coordinate. The first coordinates describe the low-frequency, connectivity end of the spectrum, and the last coordinates the high-frequency end, with $s_K(G) = \lambda_n$. When $n - 1 < K$, the positions fall between eigenvalues and neighboring coordinates interpolate between the same two eigenvalues, so they are strongly dependent; when $n - 1 > K$, the positions skip some eigenvalues. The number of positions K is a free parameter of the method; the value used in the experiments is given in Section V-A.

IV. FINGERPRINT OF BACKDOOR TROJAN

We call the change that a trigger makes to the descriptor (7) its spectral fingerprint. A defender observes only the incoming graph, not its clean version, so the fingerprint cannot be measured as the difference $\mathbf{s}(\tilde{G}) - \mathbf{s}(G)$. Detection must instead decide whether the descriptor of the observed graph lies outside the range of clean descriptors. The detector acts at inference time, as a filter in front of an already trained

model: a graph that it flags is rejected before it reaches the model. The poisoned training set is not cleaned, and the model is not retrained. We consider two detectors that differ in the data they need. The statistical detector (Section IV-A) uses clean graphs only and asks, coordinate by coordinate, whether a graph deviates more than clean graphs do. The model-based detector (Section IV-B), which is the proposed method, additionally uses triggered examples and learns one decision over all coordinates.

A. Statistical Detection of Backdoor Trojan

Let $\mathcal{R}$ be a reference set of clean graphs, and let μ_j and σ_j be the sample mean and standard deviation of the coordinate s_j over $\mathcal{R}$. Coordinates that are constant over $\mathcal{R}$ carry no information and are discarded, which leaves $m \leq K$ coordinates. For an observed graph G , each remaining coordinate is compared with the clean variation through a two-sided p -value under a Gaussian approximation,

$$p_j(G) = 2 \left[1 - \Phi \left(\frac{|s_j(G) - \mu_j|}{\sigma_j} \right) \right], \quad (8)$$

where Φ is the standard normal cumulative distribution function. The m p -values of G are corrected with the Holm procedure at level α (Section III-B), which does not require the tests to be independent; this matters here because neighboring coordinates are strongly dependent. The detector flags G , $\hat{b}_{\text{stat}}(G) = 1$, if at least one coordinate is rejected, that is, if the smallest p -value of G is at most α/m .

The detector uses no triggered graphs. Because the Gaussian model is only an approximation, however, α does not fix the fraction of clean graphs that are flagged; this fraction is measured on test data in Section V.

B. Model-Based Backdoor Detection

The statistical detector examines each coordinate on its own, so a fingerprint that consists of small deviations spread over many coordinates can go unnoticed. The model-based detector instead learns one decision over the whole descriptor. It is trained on clean graphs and their triggered versions, labelled $b = 0$ and $b = 1$, respectively. Each coordinate of the descriptor is standardized with the mean and standard deviation of the training graphs, which gives $\bar{s}(G)$, and an MLP g_ψ with parameters ψ is trained with the binary cross-entropy loss to output the detection score $q(G) = g_\psi(\bar{s}(G))$. The detector flags G when its score exceeds a threshold τ :

$$\hat{b}_{\text{MLP}}(G) = \begin{cases} 1, & q(G) > \tau, \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

The threshold is set on held-out clean graphs so that at most a fraction β of them is flagged. The budget β thus fixes the cost of the detector on clean inputs; setting it to the clean rejection rate of the statistical detector lets the two detectors be compared at the same cost (Section V-A). Logistic regression on $\bar{s}(G)$ serves as a linear reference: if the MLP separates the graphs better, part of the information lies in nonlinear combinations of coordinates.

TABLE I
EXPERIMENTAL CONFIGURATION.

Component	Setting
Target model	two-layer GCN, hidden dimension 64, dropout 0.5
Target training	Adam, lr 10^{-3} , weight decay 5×10^{-4} , batch 32, up to 50 epochs, early stopping patience 30
Target split	70/10/20 train/validation/test, five stratified folds
Attack budget	target class $y_t = 0$; poisoning rate $\rho = 0.10$; trigger size $k = 4$
Descriptor	normalized Laplacian; $K = 100$ positions
Statistical detector	two-sided Gaussian p -values; Holm at $\alpha = 0.05$
Detector pool split	70% of source graphs for training, 30% for threshold calibration
Model-based detectors	logistic regression; MLP with one hidden layer of 64 ReLU units
MLP optimization	Adam, lr 10^{-3} , 200 epochs, batch 256
Budget β	clean rejection rate of the statistical detector on the pool, leave-one-out

Training requires triggered examples of the attack to be detected, and a separate detector is trained for each dataset and attack. A defender does not normally hold such examples. The model-based results therefore show how much a detector can extract from the descriptor when the attack is known; they do not describe a defense against unseen attacks, and transfer across attacks is not studied in this paper.

V. NUMERICAL RESULTS

A. Simulation Setup

Table I lists the configuration. The target model is a two-layer graph convolutional network (GCN) [1]. It is trained on five stratified folds, each holding out 20% of the graphs for testing and 10% for validation, and is evaluated at its best-validation checkpoint (Appendix A). All numbers are averages over the five folds.

The detectors of a fold are built only from the graphs outside its test split, called the pool, so no test graph influences a detector. The descriptor uses $K = 100$ positions. This value is a fixed design choice; its influence is not studied in this paper. Since at least 93% of the graphs in every dataset have $n \leq K$ nodes, the descriptor interpolates between eigenvalues for most graphs, which is the case of strongly dependent neighboring coordinates noted in Section III-E. The reference set $\mathcal{R}$ of the statistical detector contains all clean pool graphs. For the logistic regression and the MLP, the pool is split by source graph, so that a clean graph and its triggered version always fall on the same side; the first part provides the training pairs, and the clean graphs of the second part set the threshold τ . The budget β is the rate at which the statistical detector flags clean pool graphs, each scored against a reference set from which it is left out. Both detectors therefore operate at approximately the same cost on clean graphs.

Triggered test graphs are obtained by applying the trigger to the clean test graphs, so the two sets differ only by the trigger. A triggered graph is *eligible* if its true label differs from y_t and the trigger can be applied to it. The clean accuracy (CA) is the accuracy of the poisoned model on clean test graphs, and the attack success rate (ASR) is the fraction of eligible

triggered test graphs that it assigns to y_t . For a detector, the false-positive rate (FPR) is the fraction of clean test graphs it flags, and the true-positive rate (TPR) is the fraction of eligible triggered test graphs it flags. The area under the receiver operating characteristic curve (ROC-AUC) measures how well a detector’s score ranks triggered above clean graphs over all thresholds; 0.5 corresponds to chance and 1 to perfect separation. The score is $q(G)$ for the MLP, the corresponding output for logistic regression, and the smallest Holm-adjusted p -value for the statistical detector. For the defense, every eligible triggered test graph is first passed to the detector, and the graphs it flags are rejected. The post-detection ASR is the fraction of *all* eligible triggered test graphs that are accepted and assigned to y_t by the same poisoned model; a rejected graph counts as a failed attack. It is measured by running the model on the accepted graphs rather than derived from the TPR, because the triggered graphs that a detector misses need not be those on which the attack succeeds.

1) *Datasets*: AIDS, MUTAG, and NCI1 contain small molecules and are called the molecular datasets below. PROTEINS contains protein structures; its graphs are larger and have a higher average degree. All four come from the TU-Dataset benchmark [22]; Appendix B summarizes their structure (Table III).

2) *Backdoor Trojan Attacks*: Each dataset is poisoned with SBA and Motif-Backdoor (static triggers, Section III-D1) and with GTA (dynamic trigger, Section III-D2), at the poisoning rate and trigger size of Table I. All three modify existing nodes; none adds nodes to a graph. None of the attacks is adapted to the detectors.

B. Simulation Results

Table II collects all results and answers the three questions of the introduction. The columns of the statistical detector show whether a trigger moves the descriptor beyond the clean variation (Section V-B1); the logistic-regression and MLP columns show whether the descriptor as a whole separates triggered from clean graphs (Section V-B2); and the last column group shows how much attack success remains after rejection (Section V-B3). Before any defense, the poisoned models reach a clean accuracy of 0.672–0.903. GTA succeeds on nearly every eligible graph (ASR 0.951–1.000), whereas SBA and Motif-Backdoor succeed less often (ASR 0.086–0.518) and only rarely on AIDS and MUTAG (0.086–0.152).

1) *Statistical Detection*: This part asks whether a trigger moves the descriptor beyond the variation of clean graphs. Since each triggered test graph differs from a clean test graph only by its trigger, a TPR of the statistical detector above its FPR means that this happens for the flagged graphs. The multiple-testing correction is essential for this reading. Without it, the coordinatewise test flags 30.8%–64.3% of clean test graphs, because each graph is tested at many coordinates; with the Holm correction, the clean rejection falls to 3.1%–9.2%. At this cost, the detector flags most triggered graphs only on MUTAG (68.8%–96.8%) and under GTA on AIDS (76.0%); on NCI1, GTA also gives its highest TPR, 42.5%. Under SBA and Motif-Backdoor on AIDS and NCI1, it flags

15.6%–32.8% of triggered graphs, which is 2.3 to 5.7 times its clean rejection. On PROTEINS, the TPR of 7.6%–11.8% is 2.5 to 3.9 times the FPR of 3.1%, but the ROC-AUC remains low, 0.553–0.603 against 0.5 for chance. Measured one coordinate at a time against the clean variation, the trigger is thus visible for most triggered graphs only on MUTAG and under GTA on AIDS; in the remaining settings, most triggered graphs have no single coordinate that deviates enough to survive the correction. Whether the descriptor as a whole carries more information is the subject of the next part.

2) *MLP Detection Performance*: The model-based detectors show that the trigger leaves a clearer fingerprint than the coordinatewise test reveals. The MLP reaches ROC-AUC values of 0.844–1.000 and, at its calibrated threshold, flags 34.2%–100.0% of triggered graphs at an FPR of 3.1%–9.7%; at matched clean cost, its TPR exceeds that of the statistical detector in every setting. Part of this gain comes from supervision alone: logistic regression already exceeds the ROC-AUC of the statistical detector in every setting. The MLP improves on logistic regression in 9 of the 12 settings, most under Motif-Backdoor on PROTEINS (0.761 to 0.844) and NCI1 (0.935 to 0.990), so part of the information lies in nonlinear combinations of coordinates. On MUTAG, both separate the graphs almost perfectly, and logistic regression is slightly higher under SBA; the MLP does not uniformly dominate the linear reference.

PROTEINS remains the most difficult dataset. At an FPR of 3.1%–3.2%, the MLP flags 34.2%–49.8% of triggered graphs, against 7.6%–11.8% for the statistical detector; logistic regression reaches ROC-AUC 0.761–0.842, and the MLP adds 0.04–0.08. Since no single coordinate separates most triggered PROTEINS graphs from clean ones, weak deviations spread over the descriptor remain jointly informative once labelled examples are available. PROTEINS graphs are also larger and denser than the molecular ones (Table III), so a four-node trigger changes a smaller part of the graph. This is a possible contributor to the weaker detection, not a cause established by these experiments.

3) *Analyzing Overall Defense Capability*: The last column group of Table II shows the effect of rejecting flagged graphs, and it follows the separability reported above. Under GTA, the MLP lowers the ASR from 0.999 to 0.001 on AIDS, from 1.000 to 0.000 on MUTAG, and from 1.000 to 0.003 on NCI1, whereas the statistical detector leaves 0.240, 0.032, and 0.575, respectively. On PROTEINS under GTA, rejection lowers the ASR from 0.951 to 0.844 with the statistical detector and to 0.462 with the MLP, so substantial backdoor behavior remains; under SBA and Motif-Backdoor on PROTEINS, the post-detection ASR stays between 0.300 and 0.489. On AIDS and MUTAG, SBA and Motif-Backdoor succeed rarely even without defense, so the absolute reductions there are small.

The post-detection ASR of the MLP is no higher than that of the statistical detector in every setting, and the two are equal only on MUTAG under Motif-Backdoor, where both reach 0.000. This comparison is descriptive: the five folds are not independent experiments, and the FPRs of the two detectors differ by less than one percentage point. Rejection thus nearly removes the backdoor where clean and triggered descriptors

TABLE II

DETECTION AND DEFENSE RESULTS FOR A GCN TARGET MODEL, AVERAGED OVER FIVE FOLDS. POISONED MODEL: CLEAN ACCURACY AND ATTACK SUCCESS RATE WITHOUT DEFENSE. SEPARABILITY: ROC-AUC OF THE STATISTICAL DETECTOR (STAT.), LOGISTIC REGRESSION (LR), AND THE MLP. DETECTION: FRACTION (%) OF CLEAN (FPR) AND TRIGGERED (TPR) TEST GRAPHS FLAGGED; THE MLP THRESHOLD IS CALIBRATED TO THE CLEAN REJECTION RATE OF THE STATISTICAL DETECTOR, WHOSE FPR DOES NOT DEPEND ON THE ATTACK BECAUSE IT USES NO TRIGGERED GRAPHS. DEFENSE: ATTACK SUCCESS RATE WHEN FLAGGED GRAPHS ARE REJECTED.

Dataset	Attack	Poisoned model		Separability (ROC-AUC)			FPR (%)		TPR (%)		ASR after rejection	
		CA	ASR	Stat.	LR	MLP	Stat.	MLP	Stat.	MLP	Stat.	MLP
AIDS	SBA	0.890	0.086	0.615	0.869	0.910		5.1	15.6	67.6	0.073	0.021
	Motif	0.896	0.090	0.733	0.977	0.993	5.8	4.9	32.8	99.4	0.065	0.000
	GTA	0.903	0.999	0.939	0.995	0.999		6.3	76.0	99.9	0.240	0.001
MUTAG	SBA	0.745	0.152	0.886	0.985	0.983		8.0	68.8	96.0	0.064	0.000
	Motif	0.750	0.152	0.985	1.000	1.000	7.5	8.0	95.2	100.0	0.000	0.000
	GTA	0.756	1.000	0.986	1.000	1.000		6.9	96.8	100.0	0.032	0.000
NCI1	SBA	0.680	0.330	0.621	0.868	0.913		8.7	21.0	76.3	0.254	0.083
	Motif	0.681	0.518	0.681	0.935	0.990	9.2	9.0	30.8	99.2	0.348	0.004
	GTA	0.672	1.000	0.747	0.977	0.996		9.7	42.5	99.7	0.575	0.003
PROTEINS	SBA	0.697	0.518	0.553	0.830	0.883		3.2	7.6	40.9	0.489	0.300
	Motif	0.704	0.484	0.553	0.761	0.844	3.1	3.1	11.8	34.2	0.449	0.307
	GTA	0.724	0.951	0.603	0.842	0.881		3.2	11.3	49.8	0.844	0.462

are well separated, and its benefit shrinks as they overlap.

VI. CONCLUSION

This paper examined whether structural backdoor triggers in graph classification remain imperceptible in the spectral domain. The answer depends on how the spectrum is examined. Tested one coordinate at a time against the variation of clean graphs, the trigger is visible for most triggered graphs only on MUTAG and under GTA on AIDS. Learned jointly over the whole descriptor, the fingerprint is much clearer: on the molecular benchmarks, a spectrum-only MLP separates triggered from clean graphs with ROC-AUC between 0.91 and 1.00 and, when used to reject inputs, removes nearly all success of the learned GTA trigger on AIDS, MUTAG, and NCI1 while rejecting 6.3%–9.7% of clean graphs. On PROTEINS, both detectors are less effective, and substantial attack success remains after rejection.

Three limitations bound these findings. First, the MLP is trained on triggered graphs of the attack it detects, so its results measure what the spectrum reveals about a known attack rather than describe a defense against unseen attacks; transfer across attacks is not addressed. Second, the attacks are not adapted to the detectors, and only a GCN target model and structural triggers are considered. Third, no non-spectral baseline is compared, so the detectors may respond to any structural change that the spectrum reflects, such as a changed number of edges. Detecting triggers without labelled examples, and against attackers that adapt to the detector, remains open.

REFERENCES

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [2] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *International Conference on Learning Representations (ICLR)*, 2019.
- [3] Z. Zhang, J. Jia, B. Wang, and N. Z. Gong, “Backdoor attacks to graph neural networks,” in *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies*, 2021, pp. 15–26.
- [4] Z. Xi, R. Pang, S. Ji, and T. Wang, “Graph backdoor,” in *30th USENIX Security Symposium*, 2021, pp. 1523–1540. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/xi>
- [5] X. Wang, R. Sun, Y. Zhang, B. Feng, D. He, L. Wang, and D. Jin, “Stealthy yet effective: Distribution-preserving backdoor attacks on graph classification,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025, arXiv:2509.26032.
- [6] F. R. K. Chung, *Spectral Graph Theory*, ser. CBMS Regional Conference Series in Mathematics. American Mathematical Society, 1997, vol. 92.
- [7] H. Zheng, H. Xiong, J. Chen, H. Ma, and G. Huang, “Motif-backdoor: Rethinking the backdoor attack on graph neural networks via motifs,” *IEEE Transactions on Computational Social Systems*, vol. 11, no. 2, pp. 2479–2493, 2024.
- [8] S. Yang, B. G. Doan, P. Montague, O. De Vel, T. Abraham, S. Camtepe, D. C. Ranasinghe, and S. S. Kanhere, “Transferable graph backdoor attack,” in *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses*, 2022, pp. 321–332.
- [9] Y. Ding, Y. Liu, Y. Ji, W. Wen, Q. He, and X. Ao, “SPEAR: A structure-preserving manipulation method for graph backdoor attacks,” in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 1237–1247.
- [10] X. Zhao, H. Wu, and X. Zhang, “Effective backdoor attack on graph neural networks in spectral domain,” *IEEE Internet of Things Journal*, vol. 11, no. 7, pp. 12 102–12 114, 2024.
- [11] S. Yang, B. G. Doan, P. Montague, O. De Vel, T. Abraham, A. Abuadba, E. Abbasnejad, S. Camtepe, D. C. Ranasinghe, and S. S. Kanhere, “Graph spectral purification for backdoor defence in graph neural networks,” *World Wide Web*, vol. 28, p. 49, 2025.
- [12] Z. Zhang, M. Lin, J. Xu, Z. Wu, E. Dai, and S. Wang, “Robustness inspired graph backdoor defense,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=trKNi4IUIP>
- [13] H. Yu, C. Ma, X. Wan, J. Wang, T. Xiang, M. Shen, and X. Liu, “DShield: Defending against backdoor attacks on graph neural networks via discrepancy learning,” in *Network and Distributed System Security Symposium*, 2025.

- [14] H. Sui, B. Chen, J. Zhang, C. Zhu, D. Wu, Q. Lu, and G. Long, “DMGNN: Detecting and mitigating backdoor attacks in graph neural networks,” 2024, arXiv preprint.
- [15] X. Yang, Y. Lai, K. Zhou, G. Li, J. Li, and H. Zhang, “GraphProt: Certified black-box shielding against backdoored graph models,” in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 2025, pp. 619–627.
- [16] C. Du, Y. Liu, X. Yu, Z. Xu, Y. Liu, and T. Li, “A2GBD: Attack-agnostic graph backdoor defense,” in *Proceedings of the ACM Web Conference 2026*, 2026, pp. 1229–1239.
- [17] M. M. Hasan, J. Wang, and S. Sadik, “SSP: Spectral–structural purification for graph backdoor defense,” 2026, sSRN working paper; not peer reviewed.
- [18] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [19] E. Dai, M. Lin, X. Zhang, and S. Wang, “Unnoticeable backdoor attacks on graph neural networks,” in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 2263–2273.
- [20] Z. Zhang, M. Lin, E. Dai, and S. Wang, “Rethinking graph backdoor attacks: A distribution-preserving perspective,” in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 4386–4397.
- [21] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, “The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains,” *IEEE Signal Processing Magazine*, vol. 30, no. 3, pp. 83–98, 2013.
- [22] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann, “TUDataset: A collection of benchmark datasets for learning with graphs,” in *ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+)*, 2020.
- [23] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [24] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations (ICLR)*, 2018.

TABLE III
GRAPH-CLASSIFICATION BENCHMARKS. AVERAGES ARE OVER ALL GRAPHS IN THE DATASET; $|\mathcal{E}|$ COUNTS UNDIRECTED EDGES, AND AVERAGE DEGREE IS THE MEAN OVER GRAPHS OF EACH GRAPH’S $2|\mathcal{E}|/n$.

Dataset	Graphs	Avg. n	Avg. $ \mathcal{E} $	Avg. deg.	Feat.	Classes
AIDS	2000	15.69	16.20	2.01	38	2
MUTAG	188	17.93	19.79	2.19	7	2
NCI1	4110	29.87	32.30	2.16	37	2
PROTEINS	1113	39.06	72.82	3.73	3	2

APPENDIX A TRAINING SETTINGS

The five folds are stratified, and within each fold a nested stratified split reserves 10% of the full dataset for validation. The split seed is 12345 and the model seed is 0. The reported results use a GCN target model at trigger size $k = 4$; runs with GraphSAGE [23] and GAT [24] target models and at other trigger sizes are not included in this paper.

The graph-attack task file configures SBA with an Erdős–Rényi pattern and Motif-Backdoor with the M44 motif. The run records store the shared budget and target settings but not the remaining attack parameters, which are implementation defaults: SBA edge probability 0.5, Motif-Backdoor candidate pool of 10 nodes, and, for GTA, two-hop adjacency and aggregated node features as generator input, topology threshold 0.5, feature threshold 0, and generator learning rate 0.01. SBA places its pattern on uniformly sampled host nodes. Motif-Backdoor ranks high-degree candidate nodes by how much the surrogate logits change when their incident edges are removed. For GTA, the generated adjacency is binarized with the topology threshold and restricted to pairs of nodes in the selected region. For a graph with fewer than k nodes, SBA inserts the pattern induced on the available nodes if there are at least two; otherwise, and for Motif-Backdoor and GTA, the graph is left unchanged.

For an isolated node, the corresponding entry of $\mathbf{D}^{-1/2}$ in (5) is set to zero, so that each isolated node contributes an eigenvalue of 1. Coordinates whose clean standard deviation over $\mathcal{R}$ is below 10^{-6} are treated as constant and discarded, and 10^{-6} is added to σ_j in (8) for numerical stability.

APPENDIX B BENCHMARK STRUCTURE

Table III summarizes the benchmarks. PROTEINS has larger graphs and a higher mean degree than the molecular benchmarks. A fixed trigger budget can therefore represent a smaller relative structural change; this is a possible contributor to its weaker detection, not a causal conclusion established by these comparisons.